

5.3 — Implement Error Propagation Strategies Across Multi-Agent Systems

Concept

When one agent in a multi-agent system fails, the failure does not stay local — it propagates. The coordinator that spawned the subagent must receive enough information to decide whether to retry, continue with partial results, or abort and surface the failure to the user. Getting this right requires three things: **structured error context**, **semantic distinctions between failure types**, and a deliberate **partial-result policy**.

Structured Error Context

A raw exception message from a subagent is nearly useless to a coordinator. The coordinator needs to know more than "something went wrong" — it needs to know *what* went wrong, *where*, and *what state was reached before the failure*. Every error that crosses an agent boundary should carry four fields:

Field	Purpose
<code>errorType</code>	Machine-readable category (<code>access_denied</code> , <code>not_found</code> , <code>timeout</code> , <code>upstream_failure</code>)
<code>sourceAgent</code>	Which subagent produced the error
<code>attemptedAction</code>	The specific operation that failed (tool name, query, step label)
<code>partialResults</code>	Any data successfully collected before the failure
<code>partialResults</code>	is the most commonly omitted field and the most valuable. If a subagent completes 80% of its work before failing, discarding that progress forces the coordinator to restart from scratch. Carrying it forward gives the coordinator options.

Access Failures vs. Empty Results

These two outcomes look identical to a coordinator that does not distinguish them, but they have completely different remediation paths:

- **Access failure** — the subagent was denied permission, hit an auth boundary, or was blocked by a policy. Retrying with the same credentials will not help. The coordinator must escalate: either surface the permission gap to the user or skip this data source with an explicit annotation.
- **Empty result** — the subagent completed successfully but found no matching data. This is not an error. The coordinator should continue normally, noting that the source returned zero records.

Conflating these two causes two failure modes: treating an empty result as an error (false alarm), or silently treating an access failure as empty (data gap that is invisible to the user).

Partial Result Handling

When a coordinator fans out to N subagents and only $K < N$ succeed, it faces a binary choice:

Fail entirely — appropriate when the missing subagents hold data that would change the answer materially. If you are aggregating financial records across five accounts and two are inaccessible, presenting a total from three accounts as complete is worse than returning an error.

Use what you have — appropriate when the failing subagents are supplementary or additive, and the result is still useful with caveats. In this case the coordinator must annotate the response: which sources succeeded, which failed, and why.

The default should lean toward **fail entire** for correctness-critical aggregations and **use what you have** for best-effort enrichment. Never silently return partial data as if it were complete.

Error Aggregation

When multiple subagents fail, the coordinator should collect all errors into an aggregate rather than stopping at the first. An aggregate lets the user or calling system see the full picture and make a single decision about remediation.

Code Example

```
// Shared error envelope that crosses agent boundaries
interface AgentError {
  errorType: "access_denied" | "not_found" | "timeout" | "upstream_failure";
  sourceAgent: string;
  attemptedAction: string;
  message: string;
  partialResults?: unknown;
}

interface AgentResult<T> {
  success: boolean;
  data?: T;
  error?: AgentError;
}

// Subagent: wraps its own errors before returning to coordinator
async function fetchUserOrders(userId: string): Promise<AgentResult<Order[]>> {
  try {
    const orders = await ordersDb.findByUser(userId);
    return { success: true, data: orders };
  } catch (err) {
    if (err instanceof PermissionError) {
      return {
        success: false,
        error: {
          errorType: "access_denied",
          sourceAgent: "fetchUserOrders",
        },
      };
    }
  }
}
```

```

        attemptedAction: "ordersDb.findByUser",
        message: `No read access to orders for user ${userId}`,
    },
};
}
return {
    success: false,
    error: {
        errorType: "upstream_failure",
        sourceAgent: "fetchUserOrders",
        attemptedAction: "ordersDb.findByUser",
        message: err instanceof Error ? err.message : String(err),
    },
};
}
}

// Coordinator: fan-out, collect, decide
interface AggregatedReport {
    succeeded: string[];
    failed: AgentError[];
    data: Record<string, unknown>;
    partial: boolean;
}

async function buildUserReport(userIds: string[]): Promise<AggregatedReport> {
    // Run all subagents in parallel – failures do not short-circuit others
    const results = await Promise.all(
        userIds.map(async (id) => ({ id, result: await fetchUserOrders(id) }))
    );

    const succeeded: string[] = [];
    const failed: AgentError[] = [];
    const data: Record<string, unknown> = {};

    for (const { id, result } of results) {
        if (result.success && result.data !== undefined) {
            succeeded.push(id);
            data[id] = result.data;
        } else if (result.error) {
            failed.push(result.error);
        }
    }

    // Fail entirely if more than half of sources failed – result would be misleading
    if (failed.length > userIds.length / 2) {
        throw new Error(

```

```

    `Too many subagent failures (${failed.length}/${userIds.length}). ` +
    `Errors: ${failed.map((e) => `${e.sourceAgent}: ${e.errorType}`).join(", ")}`
  );
}

return {
  succeeded,
  failed,
  data,
  partial: failed.length > 0,
};
}

// Caller distinguishes access failure from empty result
async function getSourceData(sourceId: string): Promise<AgentResult<Record[]>> {
  const response = await sourceAgent.query(sourceId);

  // Empty result – not an error, the source just has no records
  if (response.records.length === 0) {
    return { success: true, data: [] };
  }

  if (response.status === 403) {
    // Access failure – must not be silently treated as empty
    return {
      success: false,
      error: {
        errorType: "access_denied",
        sourceAgent: "getSourceData",
        attemptedAction: `sourceAgent.query(${sourceId})`,
        message: "Permission denied – source requires elevated credentials",
      },
    };
  }

  return { success: true, data: response.records };
}

```

Common Mistakes

1. **Returning a plain string error across agent boundaries** — A string cannot carry `errorType`, `sourceAgent`, or `partialResults`. The coordinator is forced to pattern-match on message text, which is brittle and breaks when messages change.
2. **Treating empty results and access failures identically** — Both return "nothing", but access failures require escalation while empty results require nothing at all. If you merge them, you either over-alert or

silently lose data.

3. **Stopping at the first subagent failure in a fan-out** — Aborting `Promise.all` early with a rejection means the coordinator never sees errors from the remaining agents. Collect all results first, then decide.
4. **Returning partial data without annotating it as partial** — A response that looks complete but is missing data from failed subagents is more dangerous than an explicit error. Always set a `partial` flag and include the list of failed sources.
5. **Discarding `partialResults` from a failing subagent** — If a subagent completed three of five steps before failing, that intermediate state has value. Carry it forward in the error envelope so the coordinator can decide whether to resume or discard.

References

- [Anthropic — Building Effective Agents](#)
- [Anthropic Docs — Tool Use Error Handling](#)
- [Claude Agent SDK — Multi-Agent Patterns](#)
- [MCP Tool Result Specification](#)